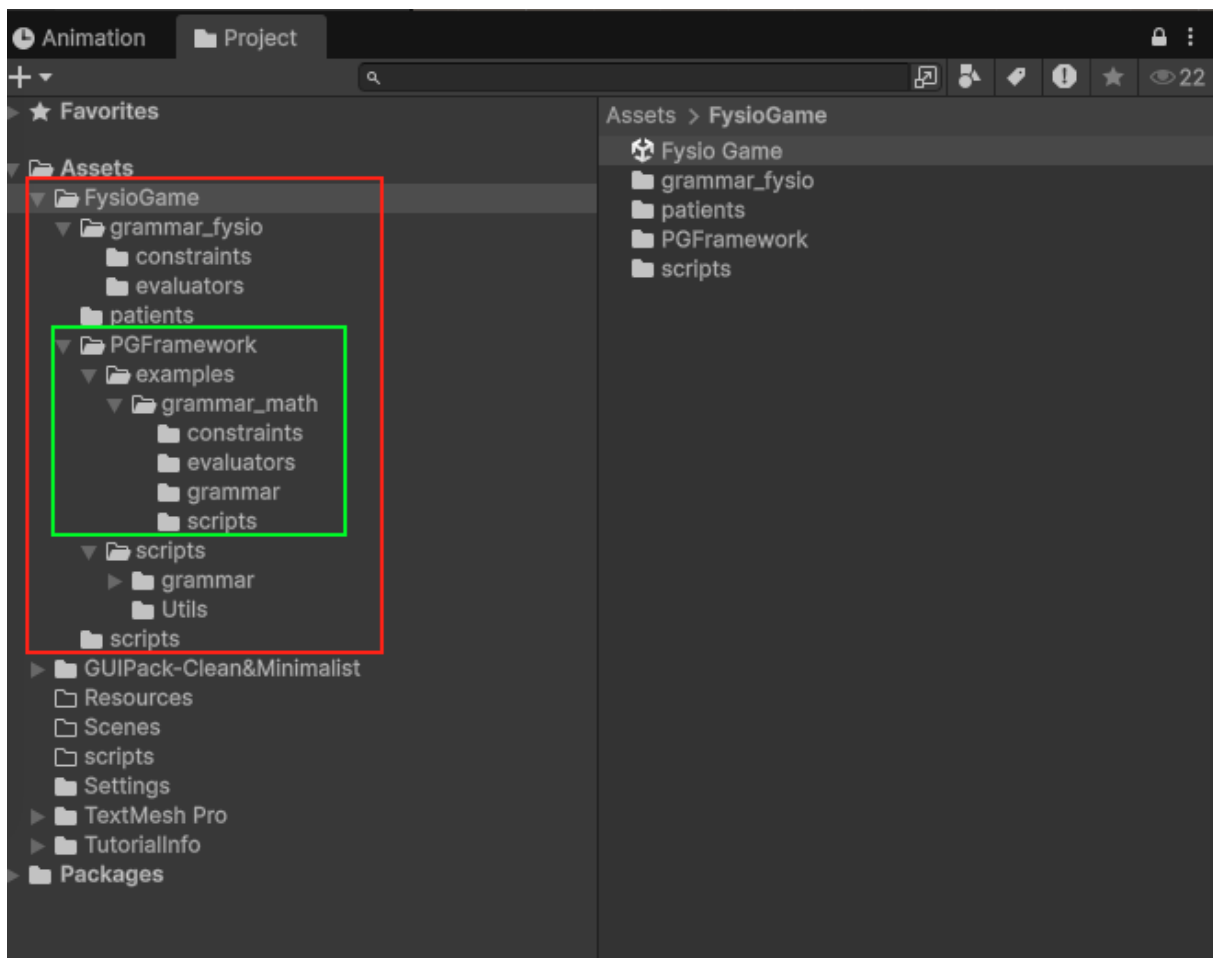


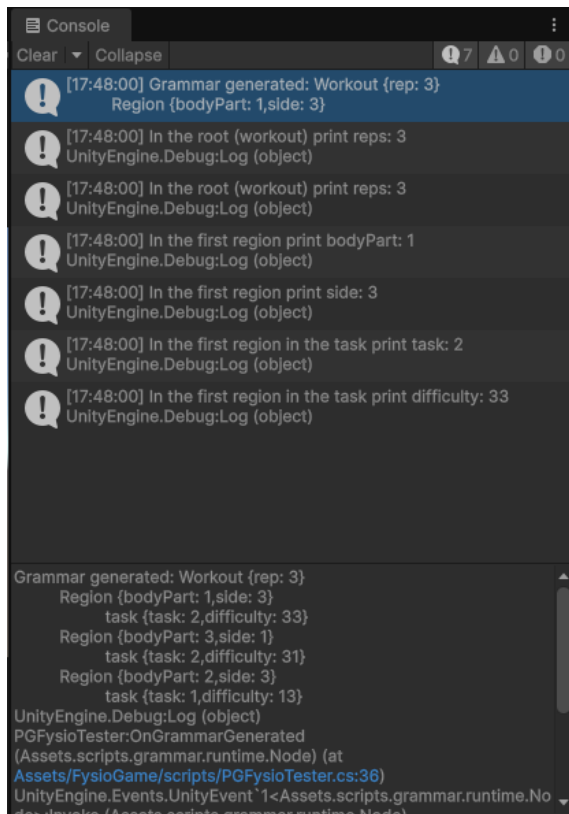
Documentation PG framework

1. Introduction

Everything that belongs to the fysio game is in the FysioGame folder (red outer rectangle). All assets that belong to the PG generation library are in the PGFramework folder (green inner rectangle). In theory you should only have to change the **PGFysioTester** script.



Run the project and see the output of the generation in the console, for example:



The output is a workout with 3 repetitions (rep). The first rep focusses on bodyPart 1, on side 3, and the player should do a task of type 2 with a difficulty of 33. See chapter **3.Generated output** for an explanation of these numbers.

The printlines below this, show how you access these elements from code (double click them in the console), or open the PGFysioTester.cs script and see the method:

```

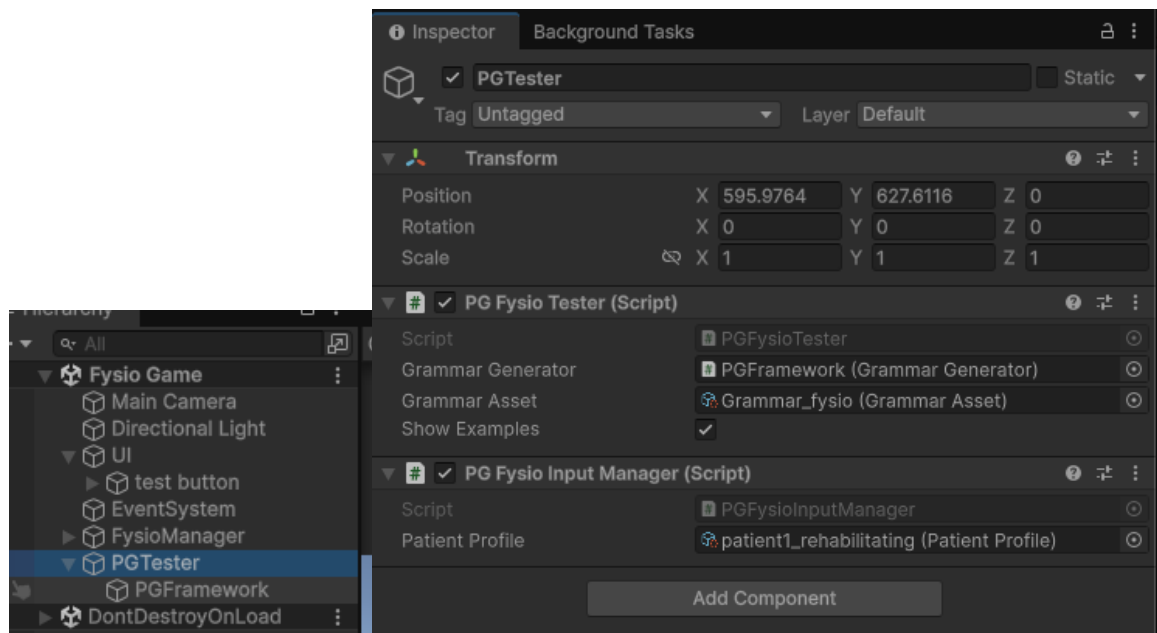
1 usage More...
private void PrintExamples(Node root)
{
    //NOTE: use the structure set by the root and its children to start building your game/app!
    // in this example the root is a Workout,
    // workout has 1 property: rep
    // workout has children of the type Region
    //     region has 2 properties: bodyPart and side
    //     region has children of the type task
    //         task has 2 properties: task and difficulty
    if (root == null) return;
    Debug.Log("In the root (workout) print reps: " + root.GetAttributeValue( name: "rep"));

    if (root.children.Count == 0) return;
    Debug.Log("In the first region print bodyPart: " + root.GetChild(0).GetAttributeValue( name: "bodyPart"));
    Debug.Log("In the first region print side: " + root.GetChild(0).GetAttributeValue( name: "side"));

    if (root.GetChild(0).children.Count == 0) return;
    Debug.Log("In the first region in the task print task: " + root.GetChild(0).GetChild(0).GetAttributeValue( name: "task"));
    Debug.Log("In the first region in the task print difficulty: " + root.GetChild(0).GetChild(0).GetAttributeValue( name: "difficulty"));
}

```

2. What scripts should you know about?

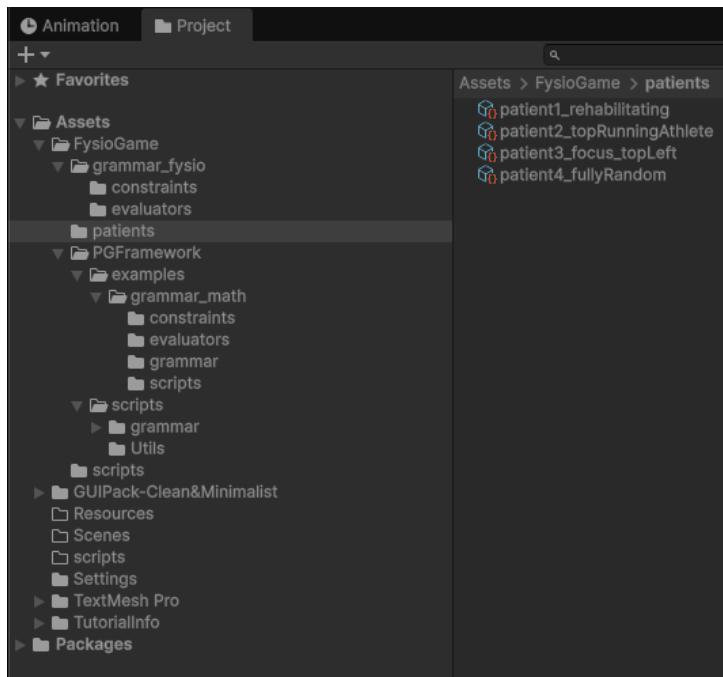


PGFysioTester: Entry point for your game generation

In the Fysio Game scene see the gameObject: PGTester -> **PGFysioTester**. This script will call the '**StartNewGeneration()**' function on the attached **GrammarGenerator**. Once the generation is done the **OnGrammarGenerated(Node root)** method in the **PGFysioTester** is called. This is your entry point to start building your specific game! You could for example call a method on one of your scripts, or use events. (It also calls the **PrintExamples(Node root)** method, this is to show examples and can be safely removed)

PGFysioInputManager: Configure the input values for the generator

The PGFysioInputManager loads a PatientProfile. This is a scriptable object in which you set the parameters for the next generation. Some patientProfiles are already made and ready to be used. You can drag one of these in the PGFysioInputManager -> Patient Profile property.



You can also modify these profiles, or create your own. See the chapter **Extra** -> **Creating your own Patient Profiles** for more info about how to create your own patient profiles.

It is recommend only changing the settings shown above, because time is limited. But if it interests you, you are ofc always free to tinker with any script or alter the Grammar that is used!

3. Generated output

When the generator is finished the **OnGrammarGenerated(Node root)** in **PGFysioTester** is called. It receives a single parameter: *root* (of type: Node). But this root has child nodes again, and these have children again, etc! This creates a tree of data. Although each node is of the type Node, it represents a specific element from the fysio domain.

The tree should look something like this (depending on your input properties it could contain more/less nodes):

Workout (0.*)→ Region (0.*)→ Task

These elements have the following properties, these all are represented as an integer(int):

Workout:

- rep: The number of repetitions (basically how many regions it will contain)

Region

This object represents a single repetition in the workout, the region contains data about what region in the body should be targeted.

- bodyPart: What body part should be targeted (0=arms, 1=legs, 2=chest)
- side: What side of the body should be targeted (0=left, 1=right, 2=center)

Task

Each Region has a single Task object. This object contains data about the type of task that should be performed for this single repetition.

- task: The type of task the patient should do for this repetition (0=follow, 1=hit, 2=dodge)
- difficulty: a number between 0-N (set in the patientProfile). Use this value to let your game set the difficulty for this specific repetition.

Tips to get started:

After generating the children, loop over the output. See the example methods **TraverseTree(..)** and **OnNodeVisited(..)** in the **PGFysioTester** class (also see the screenshot below).

When a node is visited you should do something that is specific to your game. For example, if a Region node with bodyPart:1, side:0, task: (task: 1, difficulty: 51) is visited. It could (plan to) spawn a monster (at some point) that you need to destroy with bodyPart 1 (legs) and is spawn on your left side (side 0 = left). You need to use your hammer to smack it (task 1= hit). Because the difficulty is set to 51 the monster moves rather quickly!

This ofc is just an example! Maybe in your abstract game no monsters are spawn, but the player needs to hit specific colors to mix them to something new? Use your own creativity to translate these values to a unique game experience! But make sure it still aligns with the goals for the physiotherapists (use a certain body part and side, etc).

```
/// <summary>
/// Used as an example of how to traverse the tree
/// </summary>
/// <param name="node"></param>
```

1 usage

```
private void TraverseTree(Node node)
{
    if (node == null) return;

    OnNodeVisited(node);

    foreach (var child :Node in node.children)
    {
        TraverseTree(child);
    }
}
```

```
/// <summary>
/// Used as an example of how to get the attributes of a node
/// </summary>
/// <param name="node"></param>
```

1 usage

```
private void OnNodeVisited(Node node)
{
    string result = "node: " + node.symbol.symbolName;

    result += ", attributes: ";
    List<string> attributes = node.GetAllAttributeNames(); //use node.GetAllAttributeNames() to get all the attribute names

    foreach (string attributeName in attributes)
    {
        result += attributeName + ": " + node.GetAttributeValue(attributeName) + ", "; //use node.GetAttributeValue(..) to get the value of an attribute
    }

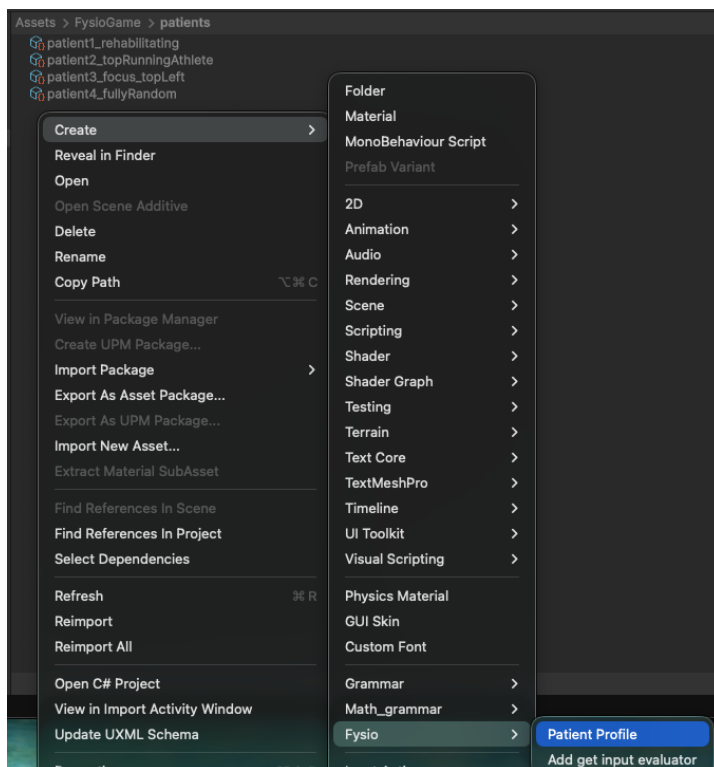
    result += "\n";
    Debug.Log(result);
}
```

4. Extra

This chapter contains optional info. It should be needed to get started

Creating your own Patient Profiles

You create a new one by right mouse clicking in a folder and 'create -> Fysio -> Patient Profile'. See below:



You can also setup your own patient profiles. You need to setup what the relative frequency is of each specific option for each property. For example, in the screenshot below: the side property has a chance to be set to 0 (weight 6), to 1 (weight 1) or to 2 (weight 3). (see chapter Generated output to see what these values mean in relation to Side).

Relative to each other this determine the chance. In this case the total weight is 10 (6+1+3). Because of that it has 60% (6/10) to become 0, 10% to become 1 (1/10) and 30% to become a 2 (3/10).

This means if all options in a property are equal it is fully random, if an option is set to 0 it is excluded.

Inspector
Background Tasks
Patient 3_focus_top Left (Patient Profile)
Open

Script
PatientProfile

▼ Body Part Frequencies
3

Element 0	1
Element 1	0
Element 2	1

+ -

▼ Side Frequencies
3

Element 0	6
Element 1	1
Element 2	3

+ -

▼ Task Frequencies
3

Element 0	1
Element 1	1
Element 2	1

+ -

Difficulty Min
40

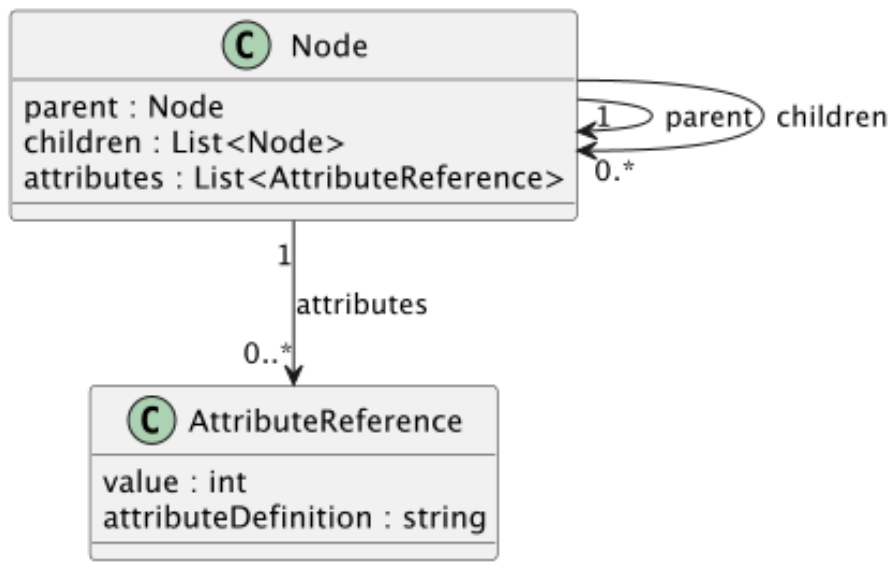
Difficulty Max
60

Number Of Repetitions_min
5

Number Of Repetitions_max
10

Node structure

The output is tree of objects from the **Node** class, connected to each other. A parent node can have child Nodes. If so, the child notes have a parent property linking back to the parent. A node can have properties, for example the 'difficulty' value. (also see the **PrintExamples(Node root)** method in the **PGFysioTester** class, on how to access these properties (or see the screenshot above).



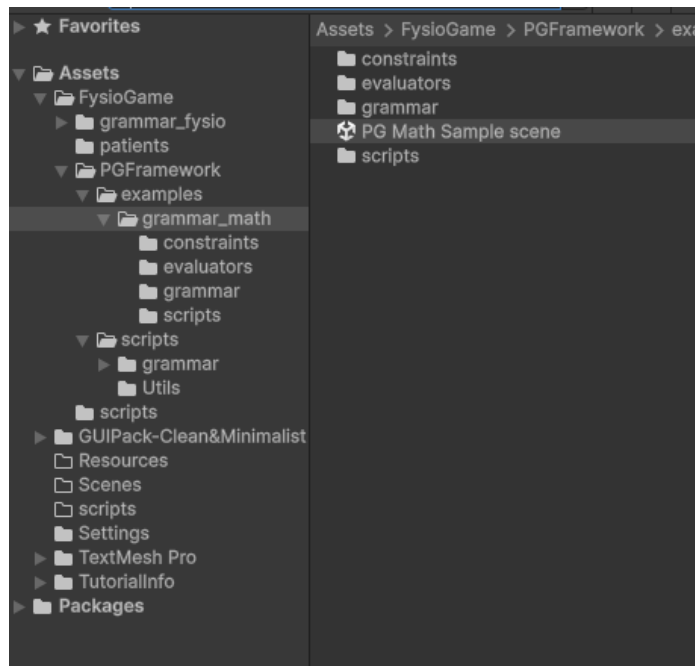
Also see the debugger below:

```

√ root = {NonTerminalNode} node (Workout)
  > attributes = {List<AttributeReference<int>>} Count = 1 Explore
  > children = {List<Node>} Count = 4 Explore
  > [0] = {NonTerminalNode} node (Region)
    > attributes = {List<AttributeReference<int>>} Count = 2 Explore
    > children = {List<Node>} Count = 1 Explore
    > [0] = {TerminalNode} node (task)
      > attributes = {List<AttributeReference<int>>} Count = 2 Explore
        > [0] = {AttributeReference<int>} task: 1
        > [1] = {AttributeReference<int>} difficulty: 26
          > attributeDefinition = {AttributeDefinition} difficulty
            > value = {int} 26
          > Raw View
        > children = {List<Node>} Count = 0 Explore
        > parent = {NonTerminalNode} node (Region)
        > symbol = {Terminal} _task (Assets.scripts.grammar.Terminal)
  
```

Math Example (extra)

To see the math example, open the 'PG Math Sample Scene'.



Run the game and open the console log

